

Relatório de Auditoria de APIs e Segurança

Projeto: CycloneDX SCA Visualizer & Dependency Tree Platform (v1.7.3)

Data da Auditoria:	31 de Agosto de 2026
Classificação:	Auditoria Técnica de APIs, OWASP API Security Top 10, Resiliência e Contratos
Avaliador / Papel:	Engenheiro Principal de APIs & Especialista em AppSec (Security-First AI Agent)
Escopo Auditado:	Interface WebMCP in-browser API, rotas RFC / Discovery (.well-known, auth.md, index.md), Nginx Gateway, Normalizador, Handlers de Parsing e Esteira GitLab CI/CD
Resultado Geral:	APROVADO COM RESSALVAS (6 Achados: 0 Críticos, 2 Altos, 3 Médios, 1 Baixo)

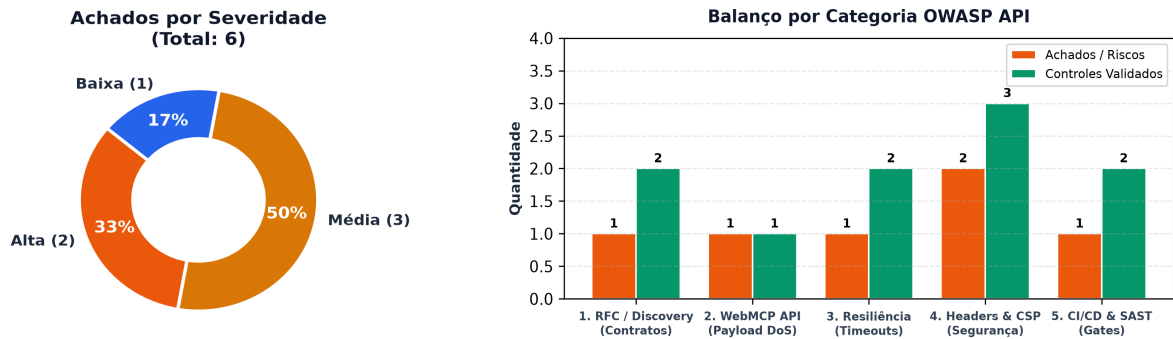
Nota Metodológica & Mapeamento de Stack para OWASP API Top 10

A arquitetura do **CycloneDX SCA Visualizer** é baseada no padrão *100% Client-Side Privacy-First*, sem persistência em banco de dados em nuvem. As categorias do **OWASP API Security Top 10 (2023)** foram mapeadas diretamente para a superfície de interfaces da aplicação:

Categoria OWASP API	Mapeamento Técnico na Stack do Projeto	Resultado da Avaliação
API1:2023 BOLA / IDOR	Consultas a recursos por ID via URL ou APIs internas. No projeto, todo o grafo de SBOM é mantido em memória RAM do navegador sem rotas /api/orders/.id.	Não Aplicável / Mitigado por Design
API2:2023 Broken Auth	Endpoints RFC e declarações de autenticação de agentes em auth.md e /.well-known/. Validação de ausência de bypass ou chaves falsas.	Conforme (Anonymous Auth Declarado)
API3:2023 BOPM / Mass Assign.	Parsing de payloads SBOM JSON/XML e mapeamento para o modelo central via normalizer.ts e Zod schemas.	Conforme (Normalização Controlada)
API4:2023 Unrestricted Resource	Limites de tamanho de payload em uploads (Dropzone) e na API in-browser WebMCP (analyze_sbom), além de Rate Limiting no Nginx.	Achados #2 e #6 (WebMCP & Nginx)
API5:2023 Broken Function Auth	Verificação de funções administrativas no cliente ou em ferramentas expostas aos agentes.	Conforme (Sem funções admin privilegiadas)
API8:2023 Security Misconfig	Cabeçalhos HTTP de segurança (CSP, CORS, HSTS), Content-Type Mismatch em endpoints RFC e CI/CD SAST gates.	Achados #1, #4 e #5

1. Resumo Executivo

A auditoria técnica de APIs, contratos e segurança da plataforma **CycloneDX SCA Visualizer (v1.7.3)** examinou a camada de roteamento Nginx, endpoints RFC de descoberta de agentes (*Agent Discovery*), ferramentas WebMCP in-browser, fluxo de sanitização de dados, chamadas HTTP de exemplos e a esteira de DevSecOps. A plataforma possui uma arquitetura robusta voltada para privacidade, porém requer ajustes de contrato MIME em rotas RFC e proteção de payload na API WebMCP.



CRÍTICA	ALTA	MÉDIA	BAIXA	PONTOS FORTES
0	2	3	1	7

2. Pontos Fortes e Riscos Centrais da API

■ Controles de Segurança e Resiliência Validados:

Controle / Proteção	Evidência Técnica no Código	Benefício de Segurança / API
Anti-XXE em Parsing XML	services/parsers/xmlParser.ts:11 processEntities: false	Mitiga exfiltração SSRF/XXE e expansão de entidades durante parsing de SBOMs XML recebidos.
Cycle Guard & Depth Cap	services/normalizer.ts:151 visited.has(ref) depth > 32	Previne DoS por Stack Overflow em grafos cíclicos manipulados injetados via arquivo ou WebMCP.
Limitação de Upload (50MB)	components/landing/Dropzone.tsx:53 file.size > 50 * 1024 * 1024	Bloqueia arquivos gigantes antes da leitura no FileReader, protegendo o heap do navegador.
Headers HTTP Hardened	nginx.conf:21-30 (HSTS, COOP, COEP, CORP, nosniff, DENY)	Defesa em profundidade rigorosa contra Clickjacking, MIME Sniffing e isolamento cross-origin.

■■ Riscos Centrais Identificados:

- Quebra de Contrato em Endpoints RFC / Discovery:** Rotas /.well-known/oauth-* respondem com Content-Type: application/json mas entregam o HTML do SPA React devido a fallback de arquivo inexistente.
- Falta de Validação de Payload no WebMCP:** A ferramenta analyze_sbom não limita tamanho de string, permitindo DoS por memória.
- Ausência de Timeout em Fetch:** Chamadas downstream em SampleLoader.tsx podem travar a UI sem AbortSignal.

3. Tabela de Achados Detalhados por Categoria

Sev.	Categoria & Arquivo:Linha	Descrição Técnica do Achado & Risco	Quick Win
ALTA	Contratos RFC & Discovery frontend/nginx.conf:92-120	Quebra de contrato JSON em endpoints RFC (.well-known): As rotas /.well-known/oauth-protected-resource e /.well-known/oauth-authorization-server possuem default_type application/json; mas usam try_files ... /index.html;. Como os arquivos JSON não existem em public/, clientes HTTP e agentes de IA recebem status 200 com HTML no corpo mas header JSON, quebrando o parser de agentes (MIME Type Mismatch).	SIM (20 min)
ALTA	WebMCP API / DoS frontend/src/utils/webMcp.ts:42-46	Ausência de limite de payload na ferramenta analyze_sbom: A API WebMCP exposta a agentes via navigator.modelContext.provideContext recebe sbomContent sem validar tamanho. O envio de payloads maliciosos de centenas de MB via API trava a thread de execução do navegador.	SIM (15 min)
MÉDIA	Resiliência Downstream frontend/src/components/landing/SampleLoader.tsx:165-173	Chamada HTTP downstream sem timeout (AbortSignal): A função loadSample executa fetch(samplePath) sem timeout configurado. Em caso de instabilidade de rede ou CDN, o estado de carregamento pode permanecer travado permanentemente.	SIM (15 min)
MÉDIA	Headers & CSP frontend/nginx.conf:30, 161	Diretiva unsafe-inline no Content-Security-Policy: O cabeçalho CSP permite style-src 'unsafe-inline'. Embora o Tailwind gere classes compiladas, o uso de unsafe-inline mitiga a proteção estrita do navegador contra injeções de estilo CSS.	NÃO (1 hr)
MÉDIA	DevSecOps & CI Gates .gitlab/ci/security.gitlab-ci.yml:18, 63	Jobs de segurança com allow_failure: true na esteira: Os jobs semgrep_sast_scan e trivy_container_scan não bloqueiam o pipeline em caso de detecção de vulnerabilidades severas, permitindo o deploy contínuo em produção.	SIM (10 min)
BAIXA	Rate Limiting frontend/nginx.conf:1-188	Ausência de limitação de taxa de requisições no Nginx: O Nginx não possui diretivas limit_req_zone para mitigar crawling abusivo ou varreduras automatizadas nos endpoints estáticos e RFC.	NÃO (45 min)

4. Recomendações Priorizadas de Remediação

Prioridade	Ação Recomendada	Impacto e Benefício
P1 (Alta)	Criar arquivos JSON reais para /.well-known/oauth-* e catalog Criar os arquivos JSON estáticos em frontend/public/.well-known/ para satisfazer rigorosamente o contrato RFC e evitar fallback para o HTML do SPA.	Garante interoperabilidade perfeita com agentes e elimina erros de parsing JSON em clientes HTTP.
P1 (Alta)	Adicionar validação de payload máximo (50MB) na API WebMCP Validar sbomContent.length <= 50 * 1024 * 1024 no método execute da tool analyze_sbom.	Elimina risco de esgotamento de memória e travamento do navegador via interface programática de agentes.
P2 (Média)	Implementar timeout resiliente com AbortSignal no fetch de exemplos Utilizar fetch(samplePath, { signal: AbortSignal.timeout(5000) }) em SampleLoader.tsx.	Assegura que requisições lentas caiam no fallback de amostras embutidas sem travar a interface.
P2 (Média)	Bloquear esteira de CI/CD para vulnerabilidades SAST e Containers Remover allow_failure: true de semgrep_sast_scan e trivy_container_scan.	Garante conformidade com os portões de qualidade de DevSecOps antes do deploy em produção.

5. Issues para o GitHub (Prontas para Criação)

Abaixo estão os templates completos formatados em Markdown para criação direta das issues de remediação de APIs e segurança no repositório:

```
--- ISSUE 1 ---
Título: [API/Segurança] Corrigir quebra de contrato e MIME type em endpoints RFC /.well-known/
Labels: api, security, severity:high, bug, nginx

### ■ Descrição do Problema
O arquivo `frontend/nginx.conf` define regras de roteamento para `/.well-known/oauth-protected-resource` e `/.well-known/oauth-authorization-server` configurando o header `Content-Type: application/json`, porém utiliza `try_files ... /index.html;`. Como os arquivos JSON correspondentes não existem em `frontend/public/.well-known/`, o Nginx responde com status HTTP 200 servindo o código HTML do SPA React. Quando agentes autônomos ou clientes HTTP realizam requisições a esses endpoints, ocorre falha de parsing JSON (Broken Contract).

### ■ Evidência no Código
Arquivo: `frontend/nginx.conf` (Linhas 92-106 e 107-121)
```nginx
location ~* ^/\.well-known/oauth-protected-resource {
 default_type application/json;
 add_header Content-Type "application/json; charset=utf-8" always;
 try_files /.well-known/oauth-protected-resource /index.html;
}
```

### ■ Impacto
Falha de interoperabilidade com agentes de IA e clientes OAuth RFC 8414 que tentam descobrir os metadados de autenticação da aplicação e recebem HTML inesperado em requisições JSON.

### ■ Sugestão de Correção
1. Criar o arquivo `frontend/public/.well-known/oauth-protected-resource` com payload JSON válido declarando `resource: "https://sca.brunoizidorio.com.br"`, `authorization_servers: []` e `scopes_supported: ["read:sbom", "analyze:sbom"]`.
2. Criar `frontend/public/.well-known/oauth-authorization-server` com metadata JSON correspondente.
3. Garantir que `try_files $uri =404;` seja aplicado a essas rotas.

### ■ Critérios de Aceite
- [ ] Requisição `curl -s -H "Accept: application/json" http://localhost:8080/.well-known/oauth-protected-resource` retorna JSON válido.
- [ ] Header `Content-Type` é `application/json; charset=utf-8`.
- [ ] Nenhum HTML do React é retornado em rotas `/.well-known`.
--- FIM ISSUE 1 ---
```

--- ISSUE 2 ---

Título: [API/Segurança] Implementar validação de tamanho máximo de payload na API WebMCP

Labels: api, security, severity:high, performance, webmcp

■ Descrição do Problema

Em `frontend/src/utils/webMcp.ts`, a ferramenta in-browser `analyze\_sbom` exposta a agentes via `navigator.modelContext.provideContext` processa o parâmetro `sbomContent` sem qualquer verificação de limite de tamanho de string (ao contrário do `Dropzone.tsx` que limita a 50MB). Um agente ou script chamando a API com strings gigantescas causa consumo excessivo de heap e travamento do navegador (Client-Side DoS).

■ Evidência no Código

Arquivo: `frontend/src/utils/webMcp.ts` (Linhas 42-46)

```
````typescript
execute: async ({ sbomContent, fileName = 'agent_input.json' }) => {
 const content = String(sbomContent);
 const name = String(fileName);
 const model = parseAndNormalizeSbom(content, name);
 ...
}
```

#### ### ■ Impacto

Exaustão de memória da aba e negação de serviço client-side caso um agente envie um arquivo SBOM desproporcional.

#### ### ■ Sugestão de Correção

Adicionar validação no início do handler `execute`:

```
````typescript
const MAX_WEBMCP_PAYLOAD = 50 * 1024 * 1024;
if (content.length > MAX_WEBMCP_PAYLOAD) {
  return { error: 'Payload excede o limite máximo permitido de 50MB.' };
}
...

```

■ Critérios de Aceite

- [] Chamada `analyze\_sbom` com conteúdo >50MB retorna objeto de erro estruturado `{ error: ... }` sem travar a thread.
- [] Testes unitários para WebMCP cobrem rejeição de payload excessivo.

--- FIM ISSUE 2 ---

--- ISSUE 3 ---

Título: [API/Performance] Configurar AbortSignal.timeout em requisições downstream de exemplos

Labels: api, performance, severity:medium, frontend

■ Descrição do Problema

Em `frontend/src/components/landing/SampleLoader.tsx`, a função `loadSample` executa chamadas `fetch(samplePath)` sem controle de timeout. Se a rede sofrer lentidão severa, a requisição fica pendente por tempo indeterminado com spinner de loading ativo.

■ Evidência no Código

Arquivo: `frontend/src/components/landing/SampleLoader.tsx` (Linhas 165-173)

■ Impacto

Experiência do usuário degradada e bloqueio de UI em conexões instáveis.

■ Sugestão de Correção

Adicionar `AbortSignal.timeout(5000)` ao `fetch`:
`const response = await fetch(samplePath, { signal: AbortSignal.timeout(5000) });`

■ Critérios de Aceite

- [] Requisições fetch em `SampleLoader` abortam automaticamente após 5 segundos e ativam o fallback embutido.

--- FIM ISSUE 3 ---

--- ISSUE 4 ---

Título: [API/Segurança] Bloquear esteira de CI/CD para falhas em scans de SAST e Container

Labels: api, security, severity:medium, devsecops, ci/cd

■ Descrição do Problema

Os jobs `semgrep\_sast\_scan` e `trivy\_container\_scan` em `.gitlab/ci/security.gitlab-ci.yml` estão com `allow\_failure: true`, permitindo que código vulnerável seja promovido para produção.

■ Sugestão de Correção

Remover `allow\_failure: true` de `semgrep\_sast\_scan` e `trivy\_container\_scan`.

--- FIM ISSUE 4 ---